

```

from zipfile import ZipFile

from pathlib import Path

sender_code = r'''// Real Time Secure Data Encryption and Decryption
// Using Video Steganography and IoT

import java.io.*;

import java.util.Scanner;

import javax.crypto.Cipher;

import javax.crypto.spec.SecretKeySpec;

import java.util.Base64;

public class VideoSteganographySender {

    static final String SECRET_KEY = "1234567890123456";

    // Encrypt Data

    public static String encrypt(String data) throws Exception {

        SecretKeySpec key = new SecretKeySpec(SECRET_KEY.getBytes(), "AES");

        Cipher cipher = Cipher.getInstance("AES");

        cipher.init(Cipher.ENCRYPT_MODE, key);

        byte[] encryptedValue = cipher.doFinal(data.getBytes());

        return Base64.getEncoder().encodeToString(encryptedValue);

    }

    // Hide encrypted text inside video file

    public static void hideDataInVideo(String videoPath, String outputPath, String message)

        throws Exception {

        FileInputStream fis = new FileInputStream(videoPath);

        FileOutputStream fos = new FileOutputStream(outputPath);

        byte[] videoBytes = fis.readAllBytes();

        fos.write(videoBytes);

        // Add separator

        fos.write("###".getBytes());

```

```

        // Add encrypted message
        fos.write(message.getBytes());

        fis.close();

        fos.close();

        System.out.println("Encrypted Message Hidden Successfully!");
    }

    public static void main(String[] args) {
        try {
            Scanner sc = new Scanner(System.in);

            System.out.println("Enter Secret Message:");

            String msg = sc.nextLine();

            String encryptedMsg = encrypt(msg);

            System.out.println("Encrypted Message:");

            System.out.println(encryptedMsg);

            hideDataInVideo(
                "input.mp4",
                "output.mp4",
                encryptedMsg);
        } catch (Exception e) {
            e.printStackTrace();
        }
    }
}

'''

receiver_code = r'''// Receiver Side - Extract + Decrypt Message

import java.io.*;

import javax.crypto.Cipher;

import javax.crypto.spec.SecretKeySpec;

import java.util.Base64;

```

```

public class VideoSteganographyReceiver {

    static final String SECRET_KEY = "1234567890123456";

    // Decrypt Data
    public static String decrypt(String encryptedData) throws Exception {

        SecretKeySpec key =
            new SecretKeySpec(SECRET_KEY.getBytes(), "AES");

        Cipher cipher = Cipher.getInstance("AES");
        cipher.init(Cipher.DECRYPT_MODE, key);

        byte[] decodedValue =
            Base64.getDecoder().decode(encryptedData);

        byte[] decryptedValue =
            cipher.doFinal(decodedValue);

        return new String(decryptedValue);
    }

    // Extract encrypted message from video
    public static String extractDataFromVideo(String videoPath)
        throws Exception {

        FileInputStream fis = new FileInputStream(videoPath);

        byte[] allBytes = fis.readAllBytes();

        fis.close();

        String content = new String(allBytes);

        String[] splitData = content.split("###");

        return splitData[1];
    }

    public static void main(String[] args) {

```

```

try {
    String encryptedMsg =
        extractDataFromVideo("output.mp4");
    System.out.println("Encrypted Message:");
    System.out.println(encryptedMsg);
    String originalMsg = decrypt(encryptedMsg);
    System.out.println("Decrypted Message:");
    System.out.println(originalMsg);
} catch (Exception e) {
    e.printStackTrace();
}
}
'''

```

readme = """# Real-Time Secure Data Encryption and Decryption Using Video Steganography and IoT
This project provides secure communication using AES encryption and video steganography techniques
in an IoT environment.

"""

```

base = Path("/mnt/data/video_steganography_project")
base.mkdir(exist_ok=True)
(sender_path := base / "VideoSteganographySender.java").write_text(sender_code)
(receiver_path := base / "VideoSteganographyReceiver.java").write_text(receiver_code)
(readme_path := base / "README.md").write_text(readme)
zip_path = "/mnt/data/video_steganography_project.zip"
with ZipFile(zip_path, 'w') as zipf:
    zipf.write(sender_path, arcname="VideoSteganographySender.java")
    zipf.write(receiver_path, arcname="VideoSteganographyReceiver.java")
    zipf.write(readme_path, arcname="README.md")
print(f"Created ZIP file: {zip_path}")

```